

Implement Playwright with TypeScript

Use Ask Mode to analyze the existing project structure and identify all impacted files required for the Playwright with TypeScript implementation.

Once the analysis is complete, switch to Agent Mode to implement the required changes

Ask mode:

Act as a senior architect and understand the entire project code. This will be useful for further enhancement or changes in the project

Architecture summary of this project

This is a Chrome MV3 browser extension that lets a user select DOM elements on any page, generate automation code prompts from that DOM, and send the prompt to an LLM provider to produce Selenium/Cucumber code. The main architectural flow is:

user opens the side panel

selects elements on the page via the inspector

selected DOM is serialized and sent to the side panel

the side panel builds prompts using language + engine + generation mode

a provider-specific API client calls the LLM

generated code is parsed and rendered back into the chat UI

1) System shape



2) Entry points and where the app boots

- Manifest and extension lifecycle: manifest.json
 - MV3 extension
 - side panel enabled via `side_panel.default_path = "panel.html"`
 - background service worker = `bg.js`
 - permissions include storage, tabs, scripting, activeTab, sidePanel
 - Background worker: `bg.js`
 - Opens the side panel when the toolbar icon is clicked
 - Listens for `SELECTED_DOM_CONTENT` messages from the content script
 - Stores the combined DOM snippet in `chrome.storage.local`
 - Performs cleanup on extension suspend
 - Side panel container: `panel.html`
 - Dual-tab UI: Code Generator and Settings
 - Loads theme config early via `init-config.js`
 - Loads UI scripts and API adapters
-

3) UI architecture

The UI is structurally split into three responsibilities:

A. Settings configuration

- `popup.js`
- Handles:
 - provider selection
 - model selection
 - API key inputs
 - saved values in `chrome.storage.sync`
 - injecting content script into the active tab
 - toggling the DOM inspector

B. Chat / generation workflow

- `chat.js`
- This is the core business logic:
 - loads provider instances from stored keys
 - listens for `SELECTED_DOM_CONTENT`

- validates provider/model before generation
- sanitizes selected DOM (removes extra <option> entries)
- selects prompt templates based on generation mode
- calls LLM APIs
- accumulates tokens and cost
- renders markdown/code output
- clears selection after generation

C. Theme/config layer

- appConfig.js
 - configUtils.js
 - Applies design tokens to CSS variables to centralize appearance
-

4) Content-script architecture

The page inspector is implemented in content.js.

This script:

- injects highlight styles into the page
- listens for TOGGLE_INSPECTOR
- adds mouseover/mouseout handlers to highlight elements
- captures click events to select DOM elements
- toggles genai-selected and genai-highlight CSS classes
- aggregates selected DOM via outerHTML
- sends SELECTED_DOM_CONTENT back to the extension

This is the key “browser-side data capture” layer. It is the bridge between the live webpage and the LLM prompt generation flow.

Important design note:

- The content script is intentionally lightweight and stateful
 - It keeps selectedElements in a Set
 - It maintains isActive, highlightedElement, and currentPort
 - Cleanup can be triggered by both messages and port disconnects
-

5) Provider abstraction layer

Each LLM provider is wrapped as a small adapter:

- groq-api.js
- openai-api.js
- inceptionlabs-api.js
- testleaf-api.js

Pattern:

- constructor accepts API key
- sendMessage(prompt, modelName) makes the API request
- returns { content, usage }
- normalizes content and usage data for UI consumption

This is a good structure for future extension, because adding a new provider is mostly a matter of creating a new adapter following the same interface.

6) Prompt design and generation strategy

Prompt generation is centralized in prompts.js.

The project currently supports:

- SELENIUM_JAVA_PAGE_ONLY
- CUCUMBER_ONLY
- CUCUMBER_WITH_SELENIUM_JAVA_STEPS

Those prompt templates explicitly instruct the model to:

- generate only Java Page Object or feature files
- constrain formatting to code blocks
- preserve deterministic output
- use specific locator selection rules

The logic for choosing prompt keys is in ChatUI.getPromptKeys() inside chat.js. It supports a checkbox-based generation mode:

- Feature file
- Page class
- or both

This is a design strength: prompt selection is decoupled from API code and centralizes generation policy.

7) Cost and token tracking

- pricing.js

This file gives a provider/model pricing map and compute function:

- calculateCost(provider, model, inputTokens, outputTokens)

The UI updates token statistics and cost display in chat.js. This is a solid pattern for observability and model governance.

8) Main runtime flow in real terms

1. User opens the extension and chooses provider/model in Settings.
 2. User clicks Inspect.
 3. Content script injects highlight handlers into the active page.
 4. User clicks elements to select DOM.
 5. content.js sends SELECTED_DOM_CONTENT via chrome.runtime.sendMessage.
 6. Background stores the snippet.
 7. Chat UI receives update and marks inspector as having content.
 8. User clicks Generate.
 9. sendMessage() builds prompt based on selected DOM and generation mode.
 10. Correct provider API adapter executes the network call.
 11. Response is parsed, rendered in markdown, and cost/token stats are updated.
-

9) Architectural strengths

- Clean separation of concerns:
 - UI: side panel
 - browser interaction: content script
 - orchestration: chat logic
 - transport/adaptor layer: API clients
 - prompt generation: templates
 - configuration: appConfig + storage
- Provider plugins are easy to add
- Good use of Chrome extension APIs and storage
- Prompt templates are deterministic and tuned for test automation generation

10) Architectural issues / real upgrade points

This is the part most useful for future enhancements:

- There is no single canonical state manager
 - Some state lives in chrome.storage
 - some in bg.js
 - some in chat.js
 - some in content.js
 - this makes the system harder to extend cleanly
- There is redundant logic for injector/connection setup
 - both popup.js and chat.js re-inject content scripts and connect ports
- The extension mixes concerns:
 - settings UI
 - generation logic
 - DOM inspection UX
 - assistant rendering
 - token cost logic
 - all in one large class
- Some provider handling is inconsistent
 - selectedProvider fallback logic in chat is permissive
 - Testleaf appears in UI but not in pricing.js
 - some provider options are listed in UI but not fully supported in pricing or prompts
- Content script selection logic is basic
 - it collects raw outerHTML
 - it does not normalize selectors or reduce DOM noise
 - this could lead to very large prompts and poor model quality
- System is currently optimized for Java/Selenium only
 - many routes check for language === 'java' && engine === 'selenium'

11) Best refactor direction for future work

If you want to improve this project without breaking it, I'd suggest the following architecture:

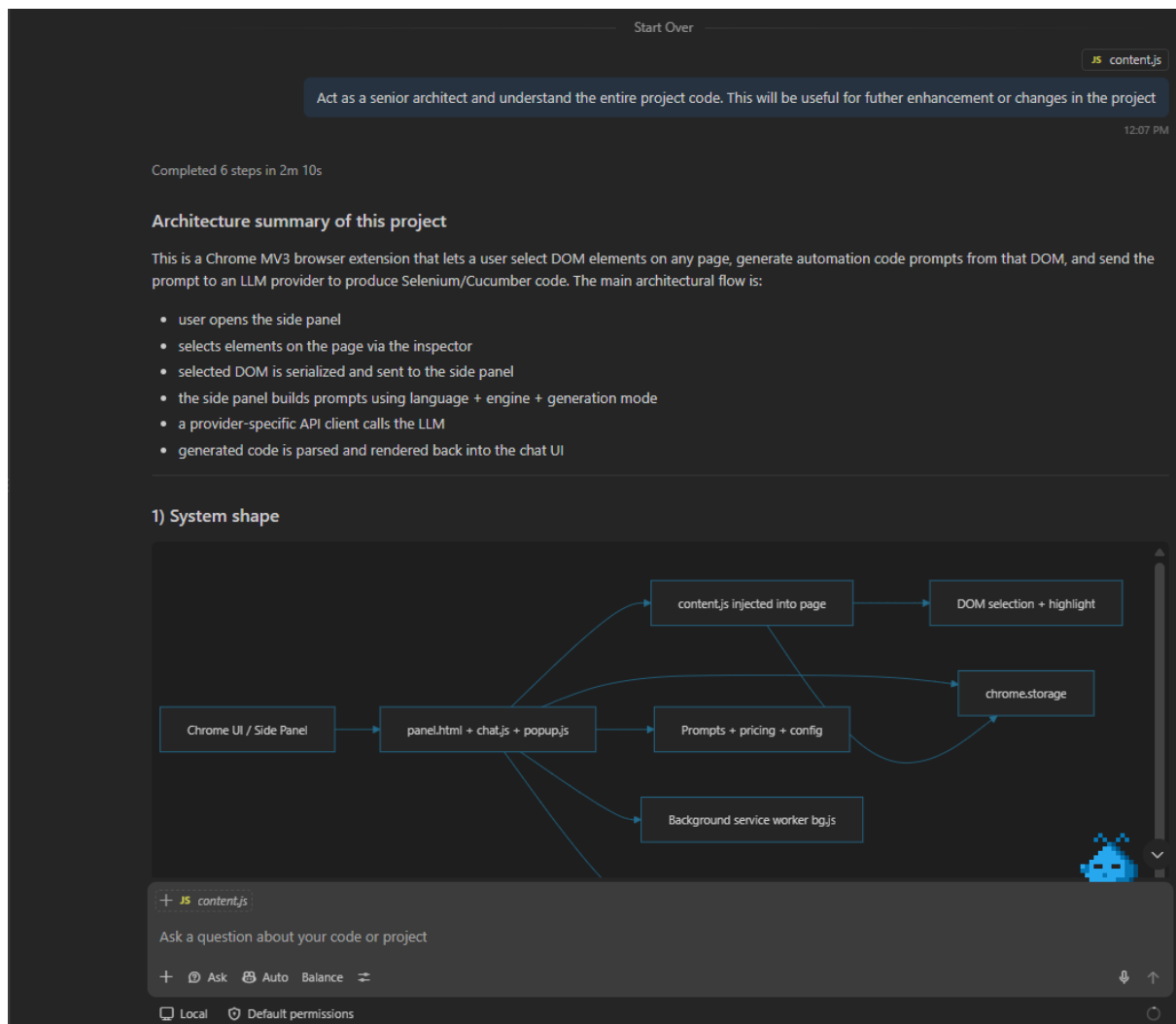
- Add a central ExtensionState module
 - provider
 - model
 - selected DOM
 - active tab
 - generation mode
 - last prompt result
- Create a uniform provider interface
 - initialize()
 - generate(prompt, model)
 - parseResponse()
 - estimateCost()
- Move DOM sanitation into a dedicated module
 - trim large nested markup
 - filter noise
 - keep useful IDs, names, data-testids, form controls
- Create a service layer for:
 - injection and lifecycle
 - messaging bus
 - cleanup and reset flows
- Keep prompt templates as the only generation-policy layer
 - avoid hardcoded checks scattered in UI code

12) My architectural judgment

This project is a strong prototype extension with a clear product use case and good separation between:

- browser interaction,
- model integration,
- prompt design,
- user experience.

It is not yet a mature platform architecture, but it is close to being an extensible MVP. The next step should be “architecture hardening,” not feature growth: centralize state, reduce duplicate initialization, and formalize the provider/service abstraction.



Plan:

Project is having implementation for Selenium with java. When i select Language as Java and Selenium as Browser Engine in the Setting tab. Selenium with Java code is getting generated successfully. I want to implement for TypeScript as Selected Language and Playwright as Browser engine without impacting the existing implementation for Selenium Java. Analyse the impacted files and functions. I need to review and approve. Then proceed with implementation in Agent mode

Output:

Review plan: TypeScript + Playwright support without disturbing Java + Selenium

I've reviewed the current generation flow and the change is isolated to the language/engine gating and prompt selection logic. The Java + Selenium path is currently treated as the

supported baseline, and the TypeScript + Playwright path can be added as a second explicit branch without changing the existing behavior.

Impacted files and functions

- chat.js
 - ChatUI.getPromptKeys()
 - ChatUI.isJavaSelenium()
 - ChatUI.addUnsupportedLanguageMessage()
 - ChatUI.sendMessage()
 - ChatUI.initializeCodeGeneratorType()
 - This is the primary decision point for supported combinations. Today it only recognizes Java + Selenium as valid for page generation; that is the exact place to add a second valid branch for TypeScript + Playwright.
- prompts.js
 - DEFAULT_PROMPTS
 - getPrompt(promptKey, variables)
 - CODE_GENERATOR_TYPES
 - This is where the generation templates live. A new prompt key for Playwright TypeScript should be added here, separate from the current Selenium Java templates.
- panel.html
 - languageBinding and browserEngine dropdowns
 - Current UI exposes Java/C# etc and Selenium/Playwright. The supported-combination guard should be kept explicit so TypeScript + Playwright is allowed while existing Java + Selenium remains unchanged.
- popup.js
 - updateModelOptions()
 - updateApiKeyVisibility()
 - model/provider selection logic
 - This file may need model options for the new Playwright path, but it should not alter the Java + Selenium flow.
- pricing.js
 - calculateCost()

- If the new TypeScript model is added to the supported model list, pricing entries may be required. This is optional only if the model is expected to show cost estimation.

What is not expected to change

- content.js is DOM capture logic and should remain unaffected.
- bg.js is background service-worker orchestration and is not part of the generation decision branch.
- ai-extension-updated/src/scripts/api/*.js should only be touched if the new flow requires a provider-specific adapter beyond the current model list.

Proposed implementation scope

I recommend the following exact scope:

1. Preserve the current Java + Selenium path as-is.
2. Add a new explicit branch when:
 - language = TypeScript
 - engine = Playwright
3. Add a dedicated Playwright TypeScript prompt template in prompts.js.
4. Update the prompt selection logic in chat.js so the new prompt is selected only for that combination.
5. Keep unsupported combinations blocked with the same validation pattern already used today.

This keeps risk low and avoids accidental regressions in the current Selenium Java feature.

Approval gate

If you approve this scope, I will proceed in Agent mode and implement the TypeScript + Playwright addition without changing the existing Selenium Java behavior.

Please confirm one of these:

- Approve as-is
- Request a narrower scope
- Suggest additional supported combinations to include in the same change

Once approved, I'll implement the change directly.

Project code review for enhancements

Act as a senior architect and understand the entire project code. This will be useful for further enhancement or changes in the project

1. a full project architecture document,
2. a refactor plan with file-by-file recommendations,
3. or a proposed modernized folder structure for this extension.

Project is having implementation for Selenium with java. When i select Language as Java and Selenium as Browser Engine in the Setting tab, Selenium with Java code is getting generated successfully. I want to implement for TypeScript as Selected Language and Playwright as Browser engine without impacting the existing implementation for Selenium Java. Analyse the impacted files and functions. I need to review and approve. Then proceed with implementation in Agent mode

Completed 3 steps in 34s

Review plan: TypeScript + Playwright support without disturbing Java + Selenium

I've reviewed the current generation flow and the change is isolated to the language/engine gating and prompt selection logic. The Java + Selenium path is currently treated as the supported baseline, and the TypeScript + Playwright path can be added as a second explicit branch without changing the existing behavior.

Impacted files and functions

- `chat.js`
 - `ChatUI.getPromptKeys()`
 - `ChatUI.isJavaSelenium()`
 - `ChatUI.addUnsupportedLanguageMessage()`
 - `ChatUI.sendMessage()`
 - `ChatUI.initializeCodeGeneratorType()`
 - This is the primary decision point for supported combinations. Today it only recognizes Java + Selenium as valid for page generation; that is the exact place to add a second valid branch for TypeScript + Playwright.
- `prompts.js`
 - `DEFAULT_PROMPTS`
 - `getPrompt(promptKey, variables)`
 - `CODE_GENERATOR_TYPES`

+ `content.js`

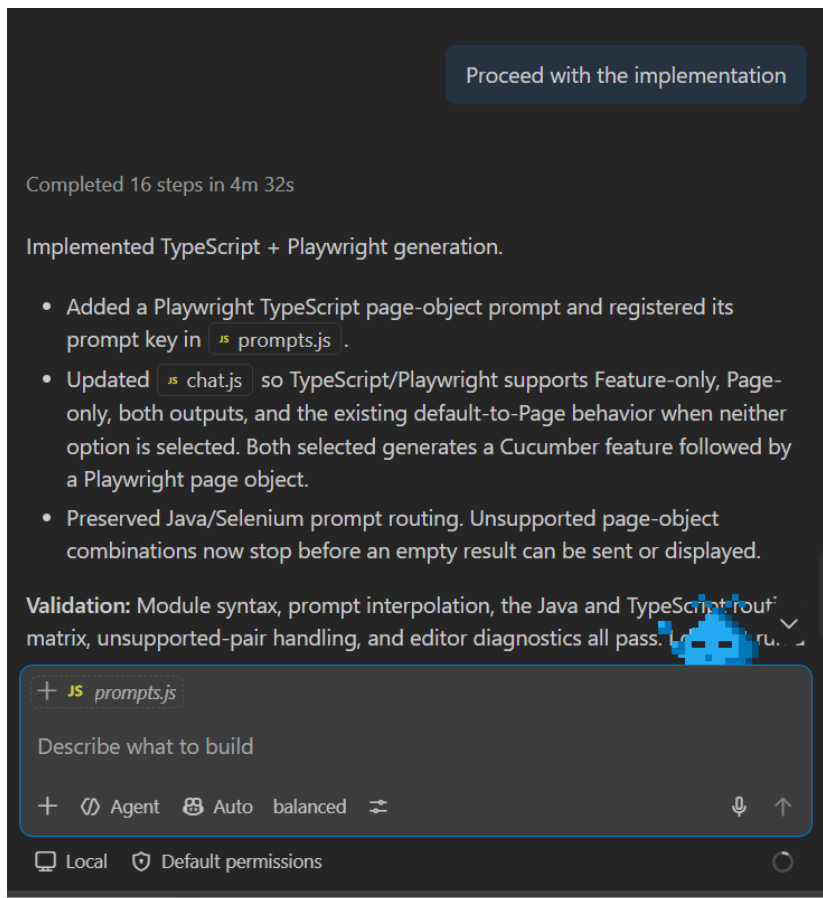
Describe what to build

+ 🔍 Agent 🛠️ Auto Balance ⚙️

📁 Local 🔒 Default permissions

Agent:

Proceed with the implementation



```
import { Page, Locator } from '@playwright/test';
```

```
export class LoginPage {
```

```
  readonly page: Page;
```

```
  readonly username: Locator;
```

```
  readonly password: Locator;
```

```
  readonly loginButton: Locator;
```

```
  constructor(page: Page) {
```

```
    this.page = page;
```

```
    this.username = page.locator('#username');
```

```
    this.password = page.locator('#password');
```

```
    this.loginButton = page.locator('input[value="Login"]');
```

```
  }
```

```
async enterUsername(value: string): Promise<void> {  
  await this.username.fill(value);  
}
```

```
async enterPassword(value: string): Promise<void> {  
  await this.password.fill(value);  
}
```

```
async clickLogin(): Promise<void> {  
  await this.loginButton.click();  
}  
}
```